Algorithmique I

Chapitre 7 · Tableaux

Corrigé des exercices

Conventions. Ces exercices reprennent le chapitre sur les tableaux [1]. Les signatures, les affectations et les boucles suivent le LDA 2026–2027. Dans les solutions, `int[n]` désigne un tableau de n entiers et `int[]` un tableau entier dont la taille est reçue à l'appel. L'ancien syllabus fixe les indices de 0 à `tab.taille` − 1. Dans les blocs de code, `<=`, `>=` et `!=` transcrivent $\leq$, $\geq$ et $\neq$. Les tableaux reçus sont initialisés. Quand l'énoncé demande une modification sur place, le paramètre tableau désigne le tableau de l'appelant. Les bornes de `for` sont inclusives ; une boucle croissante dont la borne de départ dépasse celle de fin ne s'exécute pas.

Exercice 1 : Traçons !

[1, p. 61]. Après les six affectations de la première boucle, l'indice i contient $2i$. Les deux affectations isolées lisent les valeurs déjà stockées ; la deuxième boucle ne modifie que les indices pairs.

Étape	0	1	2	3	4	5
Déclaration	?	?	?	?	?	?
Initialisation $i = 0$	0	?	?	?	?	?
$i = 1$	0	2	?	?	?	?
$i = 2$	0	2	4	?	?	?
$i = 3$	0	2	4	6	?	?
$i = 4$	0	2	4	6	8	?
$i = 5$	0	2	4	6	8	10
Après l'affectation de la case 1	0	6	4	6	8	10
Après l'affectation de la case 2	0	6	18	6	8	10
Doublement $i = 0$	0	6	18	6	8	10
$i = 2$	0	6	36	6	8	10
$i = 4$	0	6	36	6	16	10

Exercice 2 : Initialisation

[1, p. 61]. Pour chacune des trois versions, on parcourt les indices $0, \ldots, 9$. Remplacer l'affectation de la version a) par celle de b) ou de c) suffit.

```
int[] initialiser()
    int[10] tab
    int i
    for i from 0 to 9
        tab[i] = 10*(i + 1)   // a)
        // tab[i] = 2*i + 1   // b)
        // tab[i] = 10 - i    // c)
    return tab
```

Contrôle aux extrémités : a) 10 et 100 ; b) 1 et 19 ; c) 10 et 1.

Exercice 3 : Somme

[1, p. 61]. La somme des éléments déjà parcourus vaut initialement zéro ; on ajoute ensuite chaque case une fois.

```
int somme(int[] tab)
    int total, i
    total = 0
    for i from 0 to tab.taille - 1
        total = total + tab[i]
    return total
```

Exercice 4 : Maximum

[1, p. 61]. Pour un tableau non vide, on initialise le maximum à la première case, pas à zéro : cela fonctionne aussi si toutes les valeurs sont négatives.

```
int maximum(int[] tab)
    int maxi, i
    maxi = tab[0]
    for i from 1 to tab.taille - 1
        if tab[i] > maxi
            maxi = tab[i]
    return maxi
```

Exercice 5 : Minimum

[1, p. 61]. Le tableau est non vide. On conserve la plus petite valeur observée depuis la case 0.

```
int minimum(int[] tab)
    int mini, i
    mini = tab[0]
    for i from 1 to tab.taille - 1
        if tab[i] < mini
            mini = tab[i]
    return mini
```

Exercice 6 : Indice du maximum

[1, p. 62]. On mémorise l'indice du meilleur candidat ; sa valeur est `tab[indice]`. Avec $>$, une égalité ne remplace pas la première occurrence.

```
int indiceMaximum(int[] tab)
    int indice, i
    indice = 0
    for i from 1 to tab.taille - 1
        if tab[i] > tab[indice]
            indice = i
    return indice
```

Pour le dernier maximum, remplacer `>` par `>=`. Pour le premier minimum, utiliser `<` ; pour le dernier, `<=`. Dans l'exemple du cours, le premier maximum est à l'indice 4 et le dernier à l'indice 7. Le tableau doit être non vide.

Exercice 7 : Moyenne d'éléments

[1, p. 62]. On cherche les *positions* du minimum et du maximum, puis on moyenne les cases entre ces positions, bornes incluses. En cas d'ex æquo, on prend la première occurrence de chaque extrême, conformément au choix de l'exercice 6. Le tableau doit être non vide.

```
float moyenneEntreExtremes(int[] tab)
    int bas, haut, posMin, posMax, total, i
    posMin = 0
    for i from 1 to tab.taille - 1
        if tab[i] < tab[posMin]
            posMin = i
    posMax = indiceMaximum(tab)
    if posMin < posMax
        bas = posMin
        haut = posMax
    else
        bas = posMax
        haut = posMin
    total = 0
    for i from bas to haut
        total = total + tab[i]
    return total / (haut - bas + 1)
```

Dans l'exemple, on additionne 168 sur 8 cases, ce qui donne 21. Si les deux positions coïncident, le dénominateur vaut 1.

Exercice 8 : Y a-t-il un zéro ?

[1, p. 62]. On s'arrête à la première case nulle. Le test de l'indice précède l'accès à la case.

```
Boolean contientZero(int[] tab)
    int i
    i = 0
    while i < tab.taille AND tab[i] != 0
        i = i + 1
    return i < tab.taille
```

Exercice 9 : Plus grand écart absolu

[1, p. 62]. On compare chaque case à sa voisine de gauche. Le maximum porte sur les valeurs absolues, quel que soit le sens de variation.

```
int maxEcartAbsolu(int[] tab)
    int maxi, ecart, i
    maxi = 0
    for i from 1 to tab.taille - 1
        ecart = tab[i] - tab[i - 1]
        if ecart < 0
            ecart = -ecart
        if ecart > maxi
            maxi = ecart
    return maxi
```

Le plus grand écart de l'exemple est 12, entre 20 et 8. On retourne 0 par convention s'il n'y a pas de paire ; on peut aussi imposer une taille d'au moins deux cases.

Exercice 10 : Pas de négatifs !

[1, p. 63]. On change le signe seulement pour une valeur strictement négative. Le tableau est modifié sur place.

```
void rendrePositif(int[] tab)
    int i
    for i from 0 to tab.taille - 1
        if tab[i] < 0
            tab[i] = -tab[i]
```

Exercice 11 : Tableau ordonné

[1, p. 63]. On entend par ordre croissant un ordre non décroissant : deux voisins égaux sont permis. Une seule baisse suffit à conclure.

```
Boolean estCroissant(int[] tab)
    int i
    i = 1
    while i < tab.taille AND tab[i - 1] <= tab[i]
        i = i + 1
    return i >= tab.taille
```

Pour imposer un ordre *strictement* croissant, remplacer `<=` par `<`. Un tableau vide ou de taille 1 est déjà ordonné.

Exercice 12 : Recherche de valeur

[1, p. 63]. Le parcours de gauche à droite donne la première occurrence. Si toutes les cases ont été visitées sans succès, on renvoie -1.

```
int indiceValeur(int[] tab, int val)
    int i, indice
    i = 0
    while i < tab.taille AND tab[i] != val
        i = i + 1
    indice = -1
    if i < tab.taille
        indice = i
    return indice
```

Exercice 13 : Recherche dans un tableau ordonné

[1, p. 63]. La dichotomie ci-dessous cherche le premier indice dont la valeur est supérieure ou égale à la cible. La borne droite est exclue. En cas d'égalité au milieu, on continue à chercher à gauche pour obtenir la première occurrence.

```
int indiceValeurTriee(int[] tab, int val)
    int gauche, droite, milieu, indice
    gauche = 0
    droite = tab.taille
    while gauche < droite
        milieu = gauche + ((droite - gauche) DIV 2)
```

```
        if tab[milieu] < val
            gauche = milieu + 1
        else
            droite = milieu
    indice = -1
    if gauche < tab.taille AND tab[gauche] == val
        indice = gauche
    return indice
```

Sur $[1, 3, 3, 6]$, chercher 3 retourne 1 ; chercher 4 retourne -1.

Exercice 14 : Le tableau décalé

[1, p. 63]. On sauvegarde la dernière case, puis on déplace les valeurs de droite à gauche : en parcourant dans l'autre sens, on écraserait des données encore nécessaires.

```
void decalerADroite(int[] tab)
    int i, dernier
    if tab.taille > 1
        dernier = tab[tab.taille - 1]
        for i from tab.taille - 1 to 1 by -1
            tab[i] = tab[i - 1]
        tab[0] = dernier
```

Exercice 15 : Le tableau renversé

[1, p. 64]. On échange chaque case de la première moitié avec son miroir d'indice $n - 1 - i$. En taille impaire, la case centrale reste en place.

```
void renverser(int[] tab)
    int i, j, temporaire
    for i from 0 to (tab.taille DIV 2) - 1
        j = tab.taille - 1 - i
        temporaire = tab[i]
        tab[i] = tab[j]
        tab[j] = temporaire
```

Exercice 16 : Tableau symétrique

[1, p. 64]. On compare chaque case de la première moitié à son miroir, sans modifier le tableau.

```
Boolean estSymetrique(int[] tab)
    int i, moitie
    i = 0
    moitie = tab.taille DIV 2
    while i < moitie AND tab[i] == tab[tab.taille - 1 - i]
        i = i + 1
    return i == moitie
```

Exercice 17 : Égalité de tableaux

[1, p. 64]. Des tableaux de tailles différentes ne sont jamais égaux. À tailles égales, on compare les éléments occupant le même indice.

```
Boolean egaux(int[] a, int[] b)
    Boolean identiques
    int i
    identiques = a.taille == b.taille
    if identiques
        i = 0
        while i < a.taille AND a[i] == b[i]
            i = i + 1
        identiques = i == a.taille
    return identiques
```

Exercice 18 : Du nombre au tableau

[1, p. 64]. On prend les chiffres de la valeur absolue. Un premier passage détermine la taille ; le second extrait les chiffres de droite à gauche avec `MOD 10` et `DIV 10`. Zéro possède un chiffre.

```
int[] chiffres(int nombre)
    int n, copie, taille, i
    n = nombre
    if n < 0
        n = -n
    copie = n
    taille = 1
    while copie >= 10
        copie = copie DIV 10
        taille = taille + 1
    int[taille] tab
    for i from taille - 1 to 0 by -1
        tab[i] = n MOD 10
        n = n DIV 10
    return tab
```

584621 donne $[5, 8, 4, 6, 2, 1]$; 0 donne $[0]$. Le signe n'est pas un chiffre.

Exercice 19 : Le code postal

[1, pp. 64–65]. Le tableau n'est pas trié : on parcourt les communes jusqu'au premier nom correspondant. La source suppose une commune présente ; la valeur -1 protège aussi un appel avec un nom absent.

```
type Commune
    int code
    String nom

int codePostal(Commune[] tab, String nom)
    int i, code
    i = 0
    while i < tab.taille AND tab[i].nom != nom
        i = i + 1
    code = -1
    if i < tab.taille
        code = tab[i].code
    return code
```

Exercice 20 : Occurrence des chiffres

[1, p. 65]. Chaque indice d compte les occurrences du chiffre d. On réutilise l'exercice 18 pour traiter 0 sans exception supplémentaire.

```
void afficherOccurrences(int nombre)
    int[10] compte
    int[] tab
    int d, i
    for d from 0 to 9
        compte[d] = 0
    tab = chiffres(nombre)
    for i from 0 to tab.taille - 1
        d = tab[i]
        compte[d] = compte[d] + 1
    for d from 0 to 9
        if compte[d] > 0
            print d, compte[d]
```

Pour 10502851125, les couples chiffre–effectif sont $(0, 2), (1, 3), (2, 2), (5, 3), (8, 1)$.

Exercice 21 : OXO

[1, p. 65]. On teste un triplet à partir de i. Une réussite consomme les trois cases : le `O` final ne peut pas servir de premier caractère à une autre occurrence. Sinon, on décale le départ d'une case. Les symboles sont ici représentés par des chaînes d'un seul caractère.

```
void compterOXO(String[] oxo)
    int i, nombre
    i = 0
    nombre = 0
    while i + 2 < oxo.taille
        if oxo[i] == "O" AND oxo[i + 1] == "X" AND oxo[i + 2] == "O"
            nombre = nombre + 1
            i = i + 3
        else
            i = i + 1
    print nombre
```

`OXOXO` contient une occurrence comptée ; `OXOOXO` en contient deux. La chaîne d'exemple du syllabus donne 3.

Exercice 22 : Position des minimums

[1, p. 65]. Dans les deux versions, un tableau vide ne produit aucun affichage.

a) Deux parcours. Le premier trouve la valeur minimale ; le second affiche tous ses indices.

```
void afficherMinimaDeuxPasses(int[] tab)
    int i, mini
    if tab.taille > 0
        mini = minimum(tab)
        for i from 0 to tab.taille - 1
            if tab[i] == mini
                print i
```

b) Un seul parcours du tableau d'entrée. La taille logique `nb` du tableau auxiliaire représente les indices conservés. Lorsqu'un plus petit minimum apparaît, remettre `nb` à zéro suffit : les anciennes cases ne seront jamais relues.

```
void afficherMinimaUnePasse(int[] tab)
    int[tab.taille] indices
    int i, mini, nb
    nb = 0
    if tab.taille > 0
        mini = tab[0]
        for i from 0 to tab.taille - 1
            if tab[i] < mini
                mini = tab[i]
                nb = 0
            if tab[i] == mini
                indices[nb] = i
                nb = nb + 1
    for i from 0 to nb - 1
        print indices[i]
```

Sur $[4, 1, 1, 0, 2, 0]$, les indices $1, 2$ sont abandonnés quand 0 devient le minimum ; le résultat final est $3, 5$.

Exercice 23 : Les doublons

[1, p. 66]. Pour chaque position i, on ne compare qu'avec les positions suivantes : chaque paire est examinée au plus une fois.

```
Boolean contientDoublon(int[] tab)
    int i, j
    Boolean trouve
    i = 0
    trouve = false
    while i + 1 < tab.taille AND NOT trouve
        j = i + 1
        while j < tab.taille AND NOT trouve
            trouve = tab[i] == tab[j]
            j = j + 1
        i = i + 1
    return trouve
```

Exercice 24 : Mastermind

[1, p. 66]. Les deux tableaux ont la même taille. On réserve d'abord les pions bien placés. Pour chaque pion non bien placé de la proposition, on cherche ensuite un pion de même couleur dans la solution parmi les positions non réservées. Un pion de la solution n'est jamais utilisé deux fois. On ne suppose ni un ordre ni un nombre de couleurs connu.

```
void score(Couleur[] proposition, Couleur[] solution)
    Boolean[solution.taille] utilise
    int i, j, bien, mal
    bien = 0
    mal = 0
    for i from 0 to solution.taille - 1
        utilise[i] = proposition[i] == solution[i]
```

```
    if utilise[i]
        bien = bien + 1
for i from 0 to proposition.taille - 1
    if proposition[i] != solution[i]
        j = 0
        while j < solution.taille AND (utilise[j] OR
              proposition[i] != solution[j])
            j = j + 1
        if j < solution.taille
            utilise[j] = true
            mal = mal + 1
print bien, mal
```

Avec solution `ABBA` et proposition `AAAA`, on compte deux pions bien placés et zéro mal placé. Avec solution `ABBA` et proposition `BAAB`, le résultat est zéro bien placé et quatre mal placés.

Références

[1] ESI, HE2B, *Algorithmique : syllabus 2025-2026*, Bruxelles, Belgique, 2025, pp. 61-66.